DS603
Practice Set

Acknowledgement: Following problems have been collected from the Web, books "Data Structures and Algorithms – Goodrich et al", "Introduction to Algorithms – Cormen et al" and "Algorithms - Dasgupta et al". The objective of this practice set is to understand and solve them for upskilling your problem solving knowledge. It is advisable to share your thoughts with your peers while solving the practice set.

Q1. Prove that If x and y are real numbers such that $0 < x < y$ then n^x is $O(n^y)$ but n^y is not $O(n^x)$.

Q2. Given a ten-digit number n such that its first (leftmost) digit is equal to the number of 0's in n, the second digit gives the number of 1's in n, and so on. The last (rightmost) digit gives the number of 9's in n. Devise an algorithm to find n.

Q3. Devise an algorithm to find all common elements present in every row of a given matrix.

Q4. Devise an algorithm to check whether a given matrix is a unitary matrix.

Q5. Devise an algorithm to remove duplicates from a sorted linked list.

Q6. Devise an algorithm to check whether a pair of numbers exist in an array whose sum is equal to a given number.

Q7. Devise an algorithm to check whether a pair of numbers exist in an array whose product is equal to a given number.

Q8. Devise an algorithm to check whether an array of integers contains any duplicate number(s). If so, display duplicate number(s) and its frequency.

Q9. Solve and compute complexity
(a) $T(n) = 7T(n/7) + n$
(b) $T(n) = 8T(n/2) + n^3$
(c) $T(n) = T(\sqrt{n}) + 1$

Q10. Design an algorithm that, given an array A of n integers and another integer x, determines whether or not there exist three (not necessarily distinct) elements in A whose sum is equal to x.

Q11. Design an algorithm that, given an array A of n integers and another integer x, determines whether or not there exist three (not necessarily distinct) elements in A whose sum is less than or equal to x.

Q12. Devise an iterative and a recursive algorithm to calculate the sum of first n Fibonacci numbers. Comprare the time and space complexity of your solution.

Q13. Devise an algorithm that, given an array A of n objects, finds the majority element or reports that no such element exists.

Q14. Devise an algorithm to find all distinct combinations of a given string of length 10.

Q15. Devise an algorithm that can generate all permutations of the first n positive integers.

Q16. Devise an algorithm to check if a given string is a rotated palindrome or not.

Q17. Devise an algorithm to reverse a given sentence without reversing the individual words.

Q18. Devise an algorithm to remove adjacent duplicate characters from a given string.

Q19. Devise an algorithm to find all n-digit numbers which match with the sum of digits.

Q20. Devise an algorithm to multiply two integers using divide-and-conquer method.

Q21. Devise an algorithm to sort a binary array.

Q22. Devise an algorithm to find number of 1’s in a sorted binary array

Q23. Given an array of n numbers, devise an algorithm to find the largest sum of a consecutive subsequence of the array (the numbers are not necessarily positive).

Q24. Devise an algorithm to merge two arrays without any constraints.

Q25. Devise an algorithm to merge two arrays with given constraints, say the resultant array is sorted, no duplicate element in the resultant array or count the number of elements in the resultant array that are equal to the product of any three elements in the resultant array.

Q26. Devise an algorithm (using divide-and-conquer) to find the minimum sum subarray of a given size n.

Q27. Devise an algorithm to find the maximum sum of subsequence with no adjacent elements.

Q28. Devise an algorithm to check whether a given expression is a balanced expression.

Q29. Devise an algorithm to evaluate the value of an arithmetic expression in reverse polish notation.

Q30. Devise an algorithm to implement a stack using a linked list.

Q31. Devise an algorithm to find duplicate parenthesis in a given expression.

Q32. Devise an algorithm to reverse a stack using a queue.

Q33. Devise an algorithm to delete the middle element from a stack.

Q34. Devise an algorithm to implement a queue using two stacks.

Q35. Devise an algorithm to implement a stack using a single queue.

Q36. Devise an algorithm to implement a stack using queues.

Q37. Devise an algorithm to sort a stack using a temporary stack.

Q38. Devise an algorithm to find the next smaller of an element in an array.

Q39. Devise an algorithm to find the next smaller of next greater element in an array.

Q40. Devise an algorithm to find a missing term in a given sequence.

Q41. Devise an algorithm to check whether a given binary tree is a complete binary tree.

Q42. Devise an algorithm to calculate the height of a binary tree.

Q43. Devise an algorithm (iterative and recursive) that traverses a tree using preorder traversal logic.

Q44. Devise an algorithm to implement priority queues using binary heaps.

Q45. Devise an algorithm to check if a given array represents a min heap or not.

Q46. Devise an algorithm to remove an arbitrary element from a heap of size n.

Q47. Devise an algorithm (iterative and recursive) for implementing depth first search.

Q48. Devise an algorithm to perform a depth first search on a binary tree.

Q49. Devise an algorithm (iterative and recursive) for implementing breadth first search

Q50. Devise an algorithm to check if an undirected graph contains cycle or not.

Q51. Devise an algorithm to check if an undirected graph is a tree.